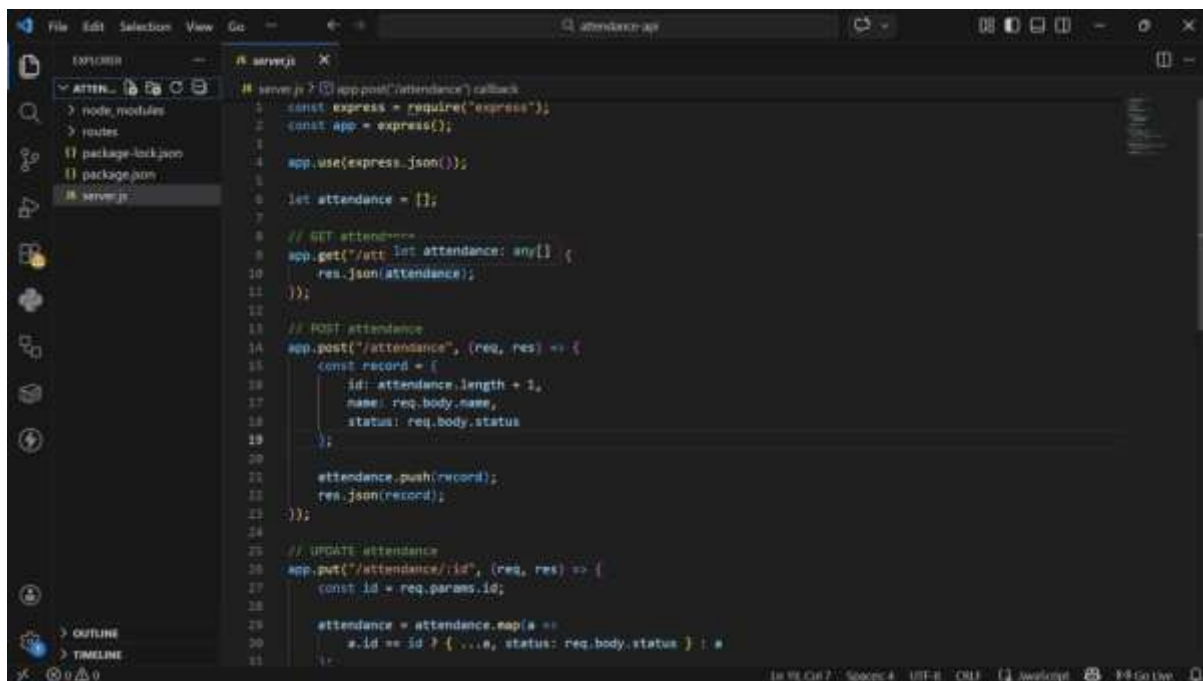


**School of Computer Science and Artificial
Intelligence**

Lab Assignment # 15.5

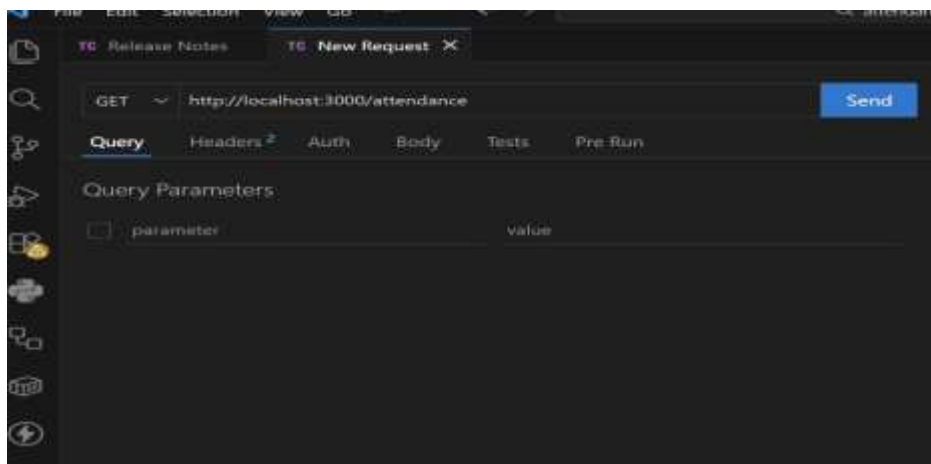
Program : B. Tech (CSE)
Specialization :AIML
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : VI
Academic Session : 2025-2026
Name of Student : G. Usha Sree
Enrollment No. : 2303A52431
Batch No. : 34
Date :13/03/26

Task 1 – Student Attendance API

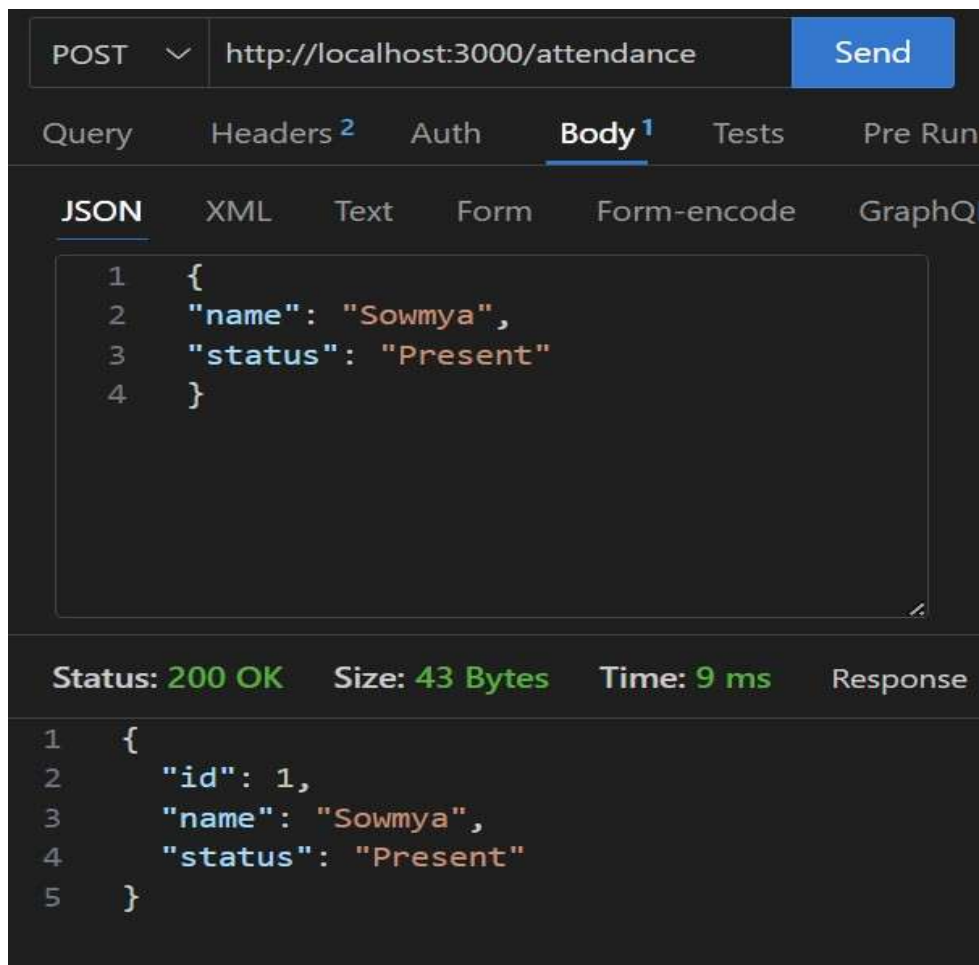


```
1 // server.js
2 const express = require("express");
3 const app = express();
4 app.use(express.json());
5
6 let attendance = [];
7
8 // GET attendance
9 app.get("/att", (req, res) => {
10   res.json(attendance);
11 });
12
13 // POST attendance
14 app.post("/attendance", (req, res) => {
15   const record = {
16     id: attendance.length + 1,
17     name: req.body.name,
18     status: req.body.status
19   };
20
21   attendance.push(record);
22   res.json(record);
23 });
24
25 // UPDATE attendance
26 app.put("/attendance/:id", (req, res) => {
27   const id = req.params.id;
28
29   attendance = attendance.map(a =>
30     a.id == id ? { ...a, status: req.body.status } : a
31   );
32 });
```

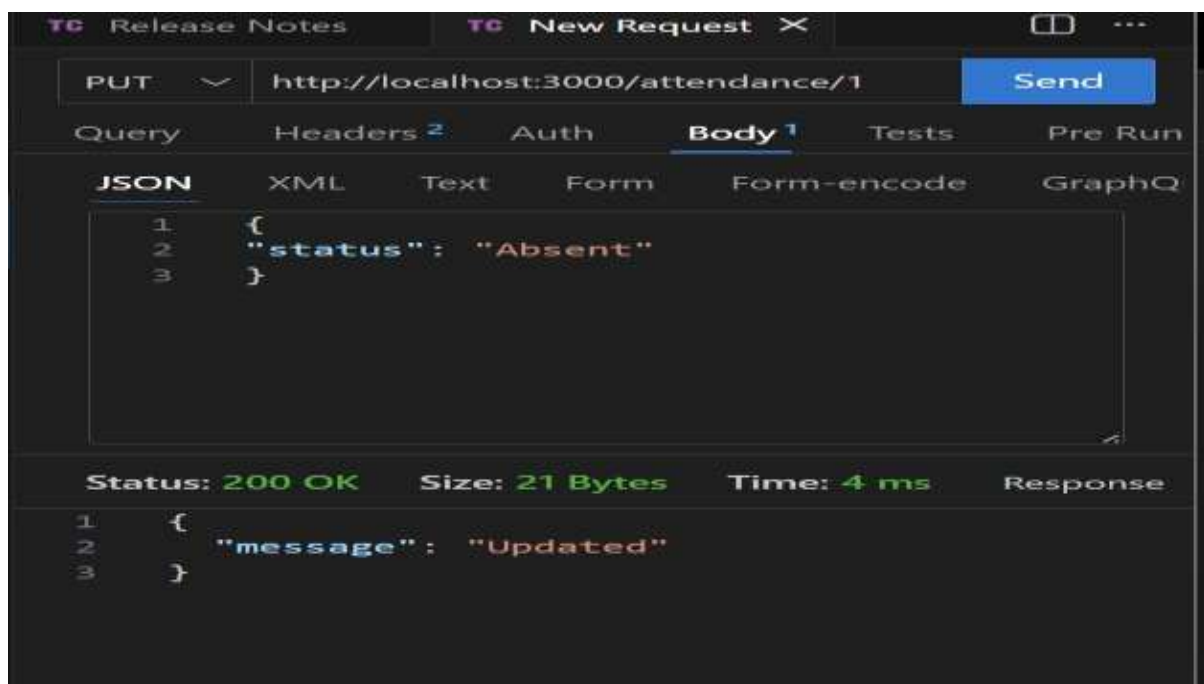
GET /attendance → View attendance records



o **POST /attendance** → **Mark attendance**



o **PUT /attendance/{id}** → **Update attendance record**



o **DELETE /attendance/{id} → Remove attendance entry**

The screenshot shows a REST client interface with a DELETE request to `http://localhost:3000/attendance/1`. The request body is a JSON object: `{ "status": "Absent" }`. The response is a 200 OK status with a JSON body: `{ "message": "Deleted" }`.

DELETE `http://localhost:3000/attendance/1` Send

Query Headers ² Auth **Body ¹** Tests Pre Run

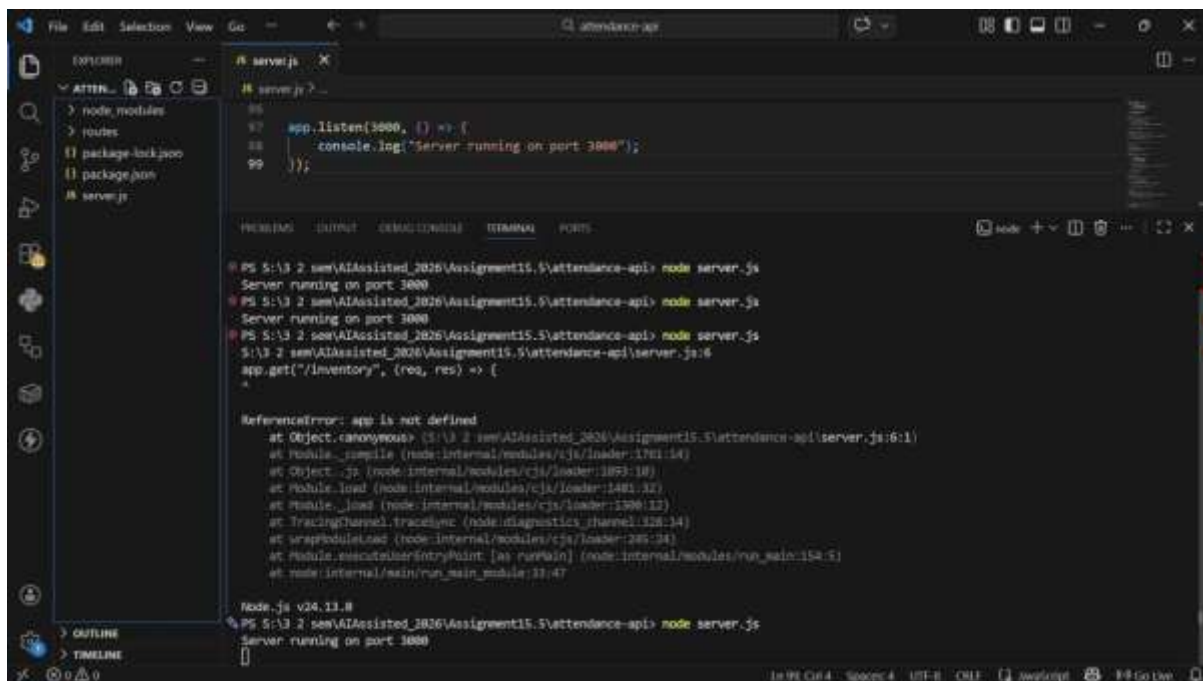
JSON XML Text Form Form-encode GraphQL

```
1 {
2   "status": "Absent"
3 }
```

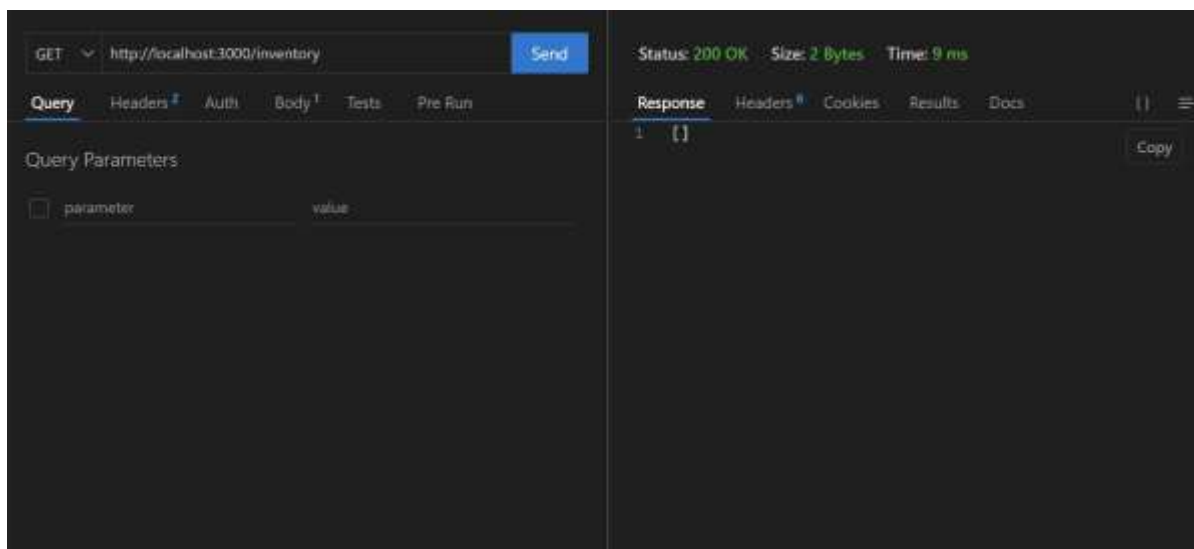
Status: **200 OK** Size: **21 Bytes** Time: **4 ms** Response

```
1 {
2   "message": "Deleted"
3 }
```

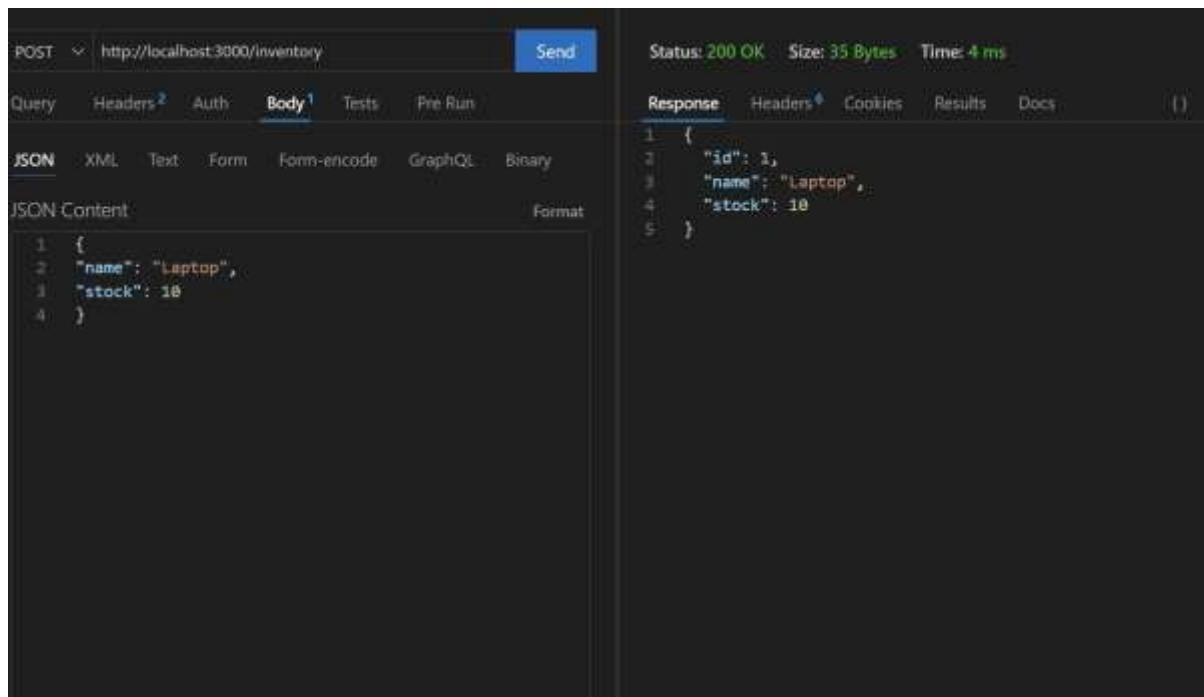
Task 2 – Inventory Management API



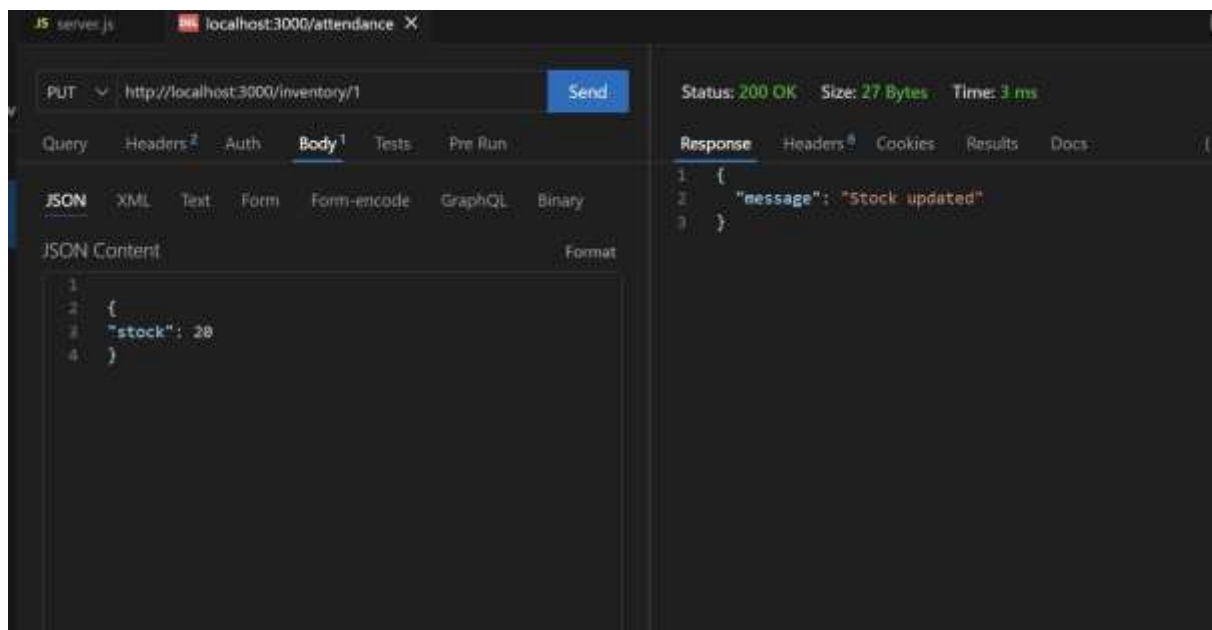
GET /inventory → List all items



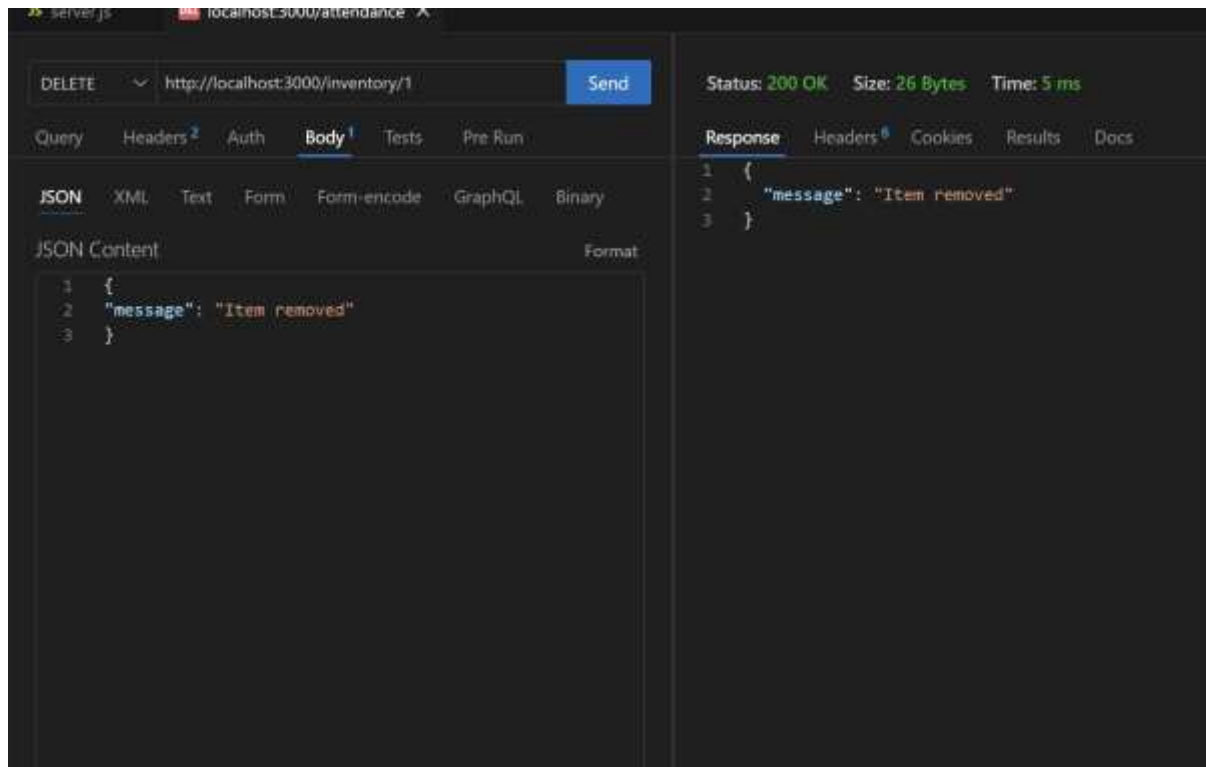
- o **POST /inventory** → **Add new item**



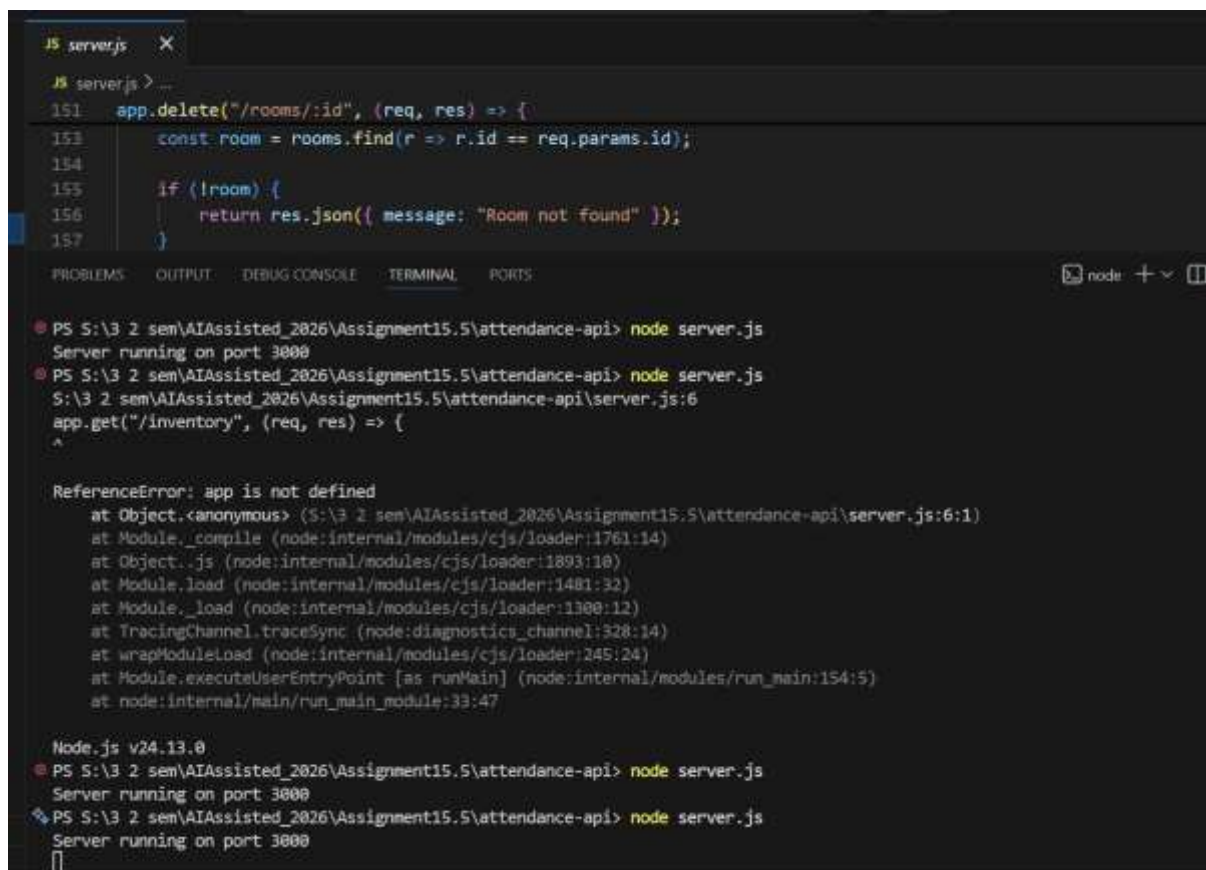
o **PUT /inventory/{id} → Update stock quantity**



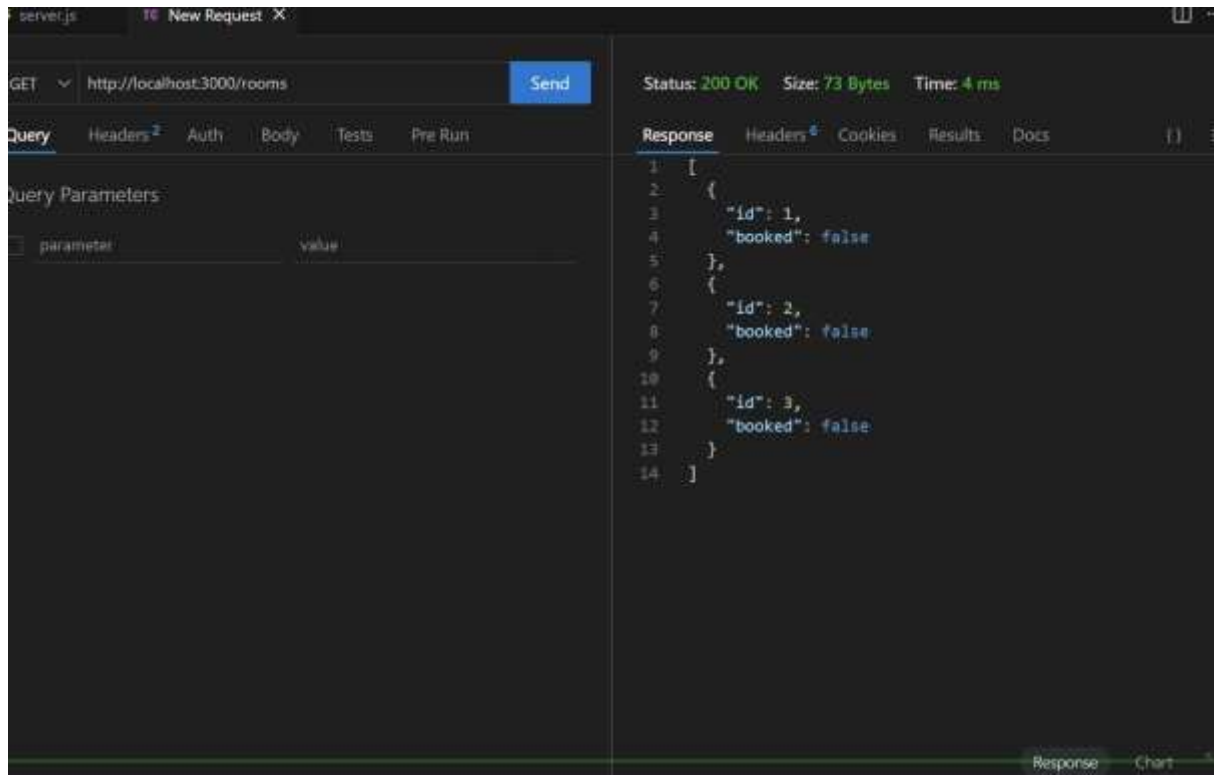
o **DELETE /inventory/{id} → Remove an item**



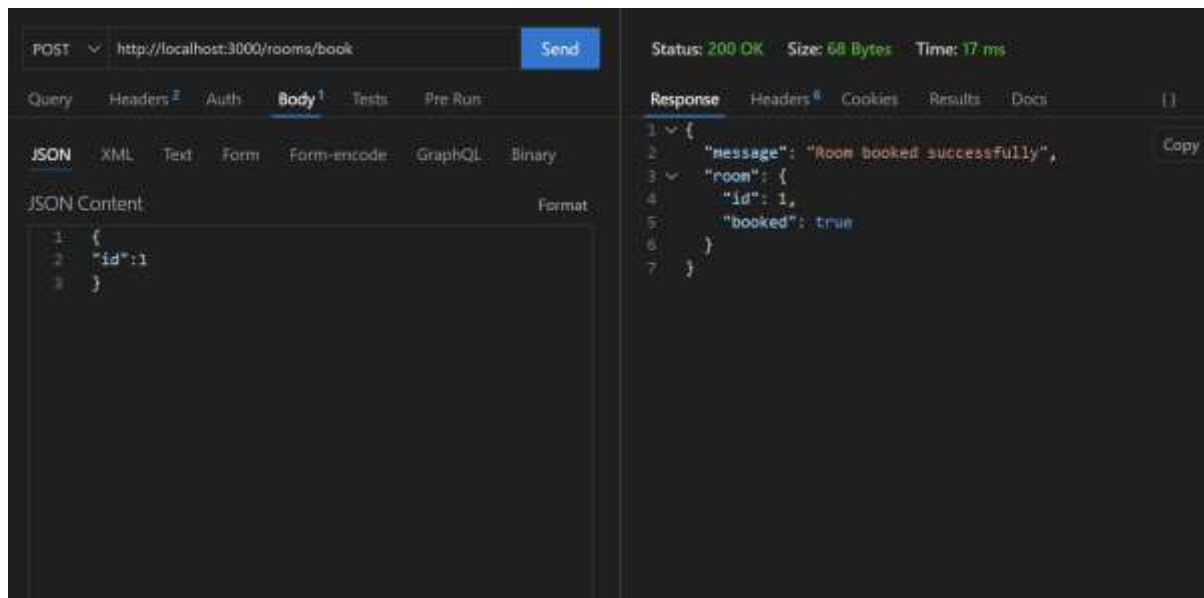
Task 3 – Hotel Room Booking API



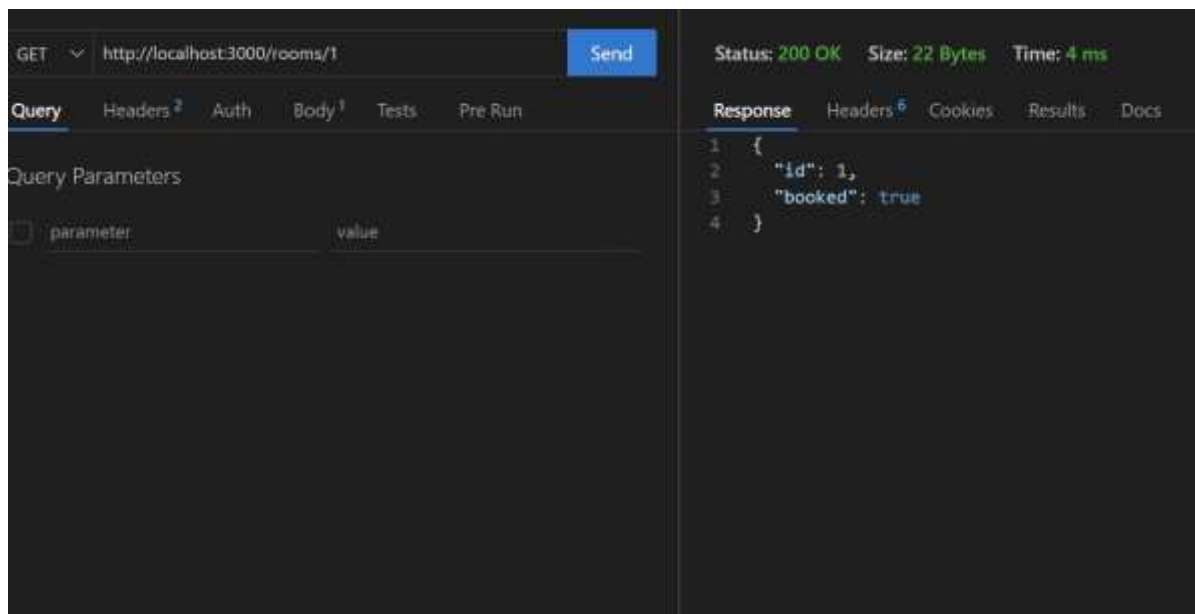
o GET /rooms → View available rooms



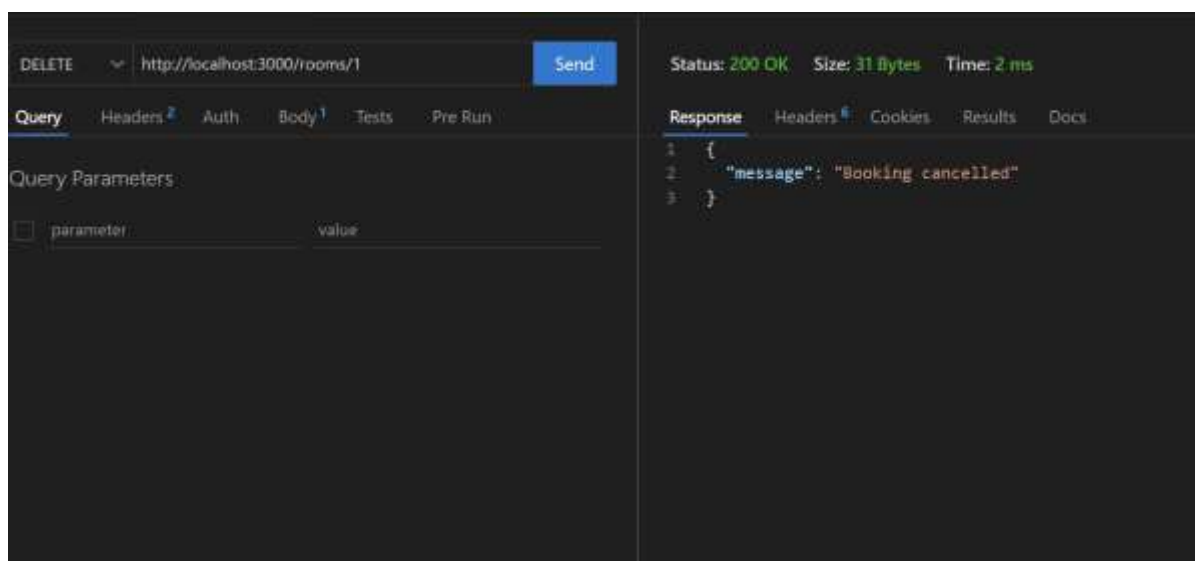
o POST /rooms/book → Book a room



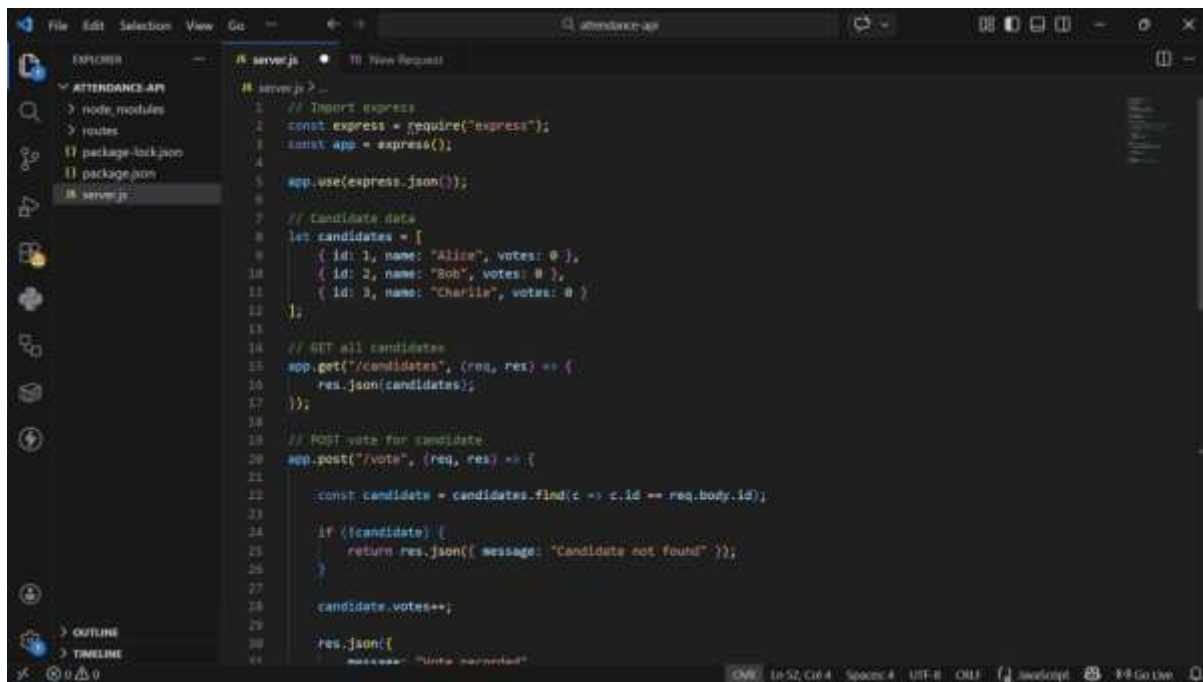
o GET /rooms/{id} → View booking details



o DELETE /rooms/{id} → Cancel booking

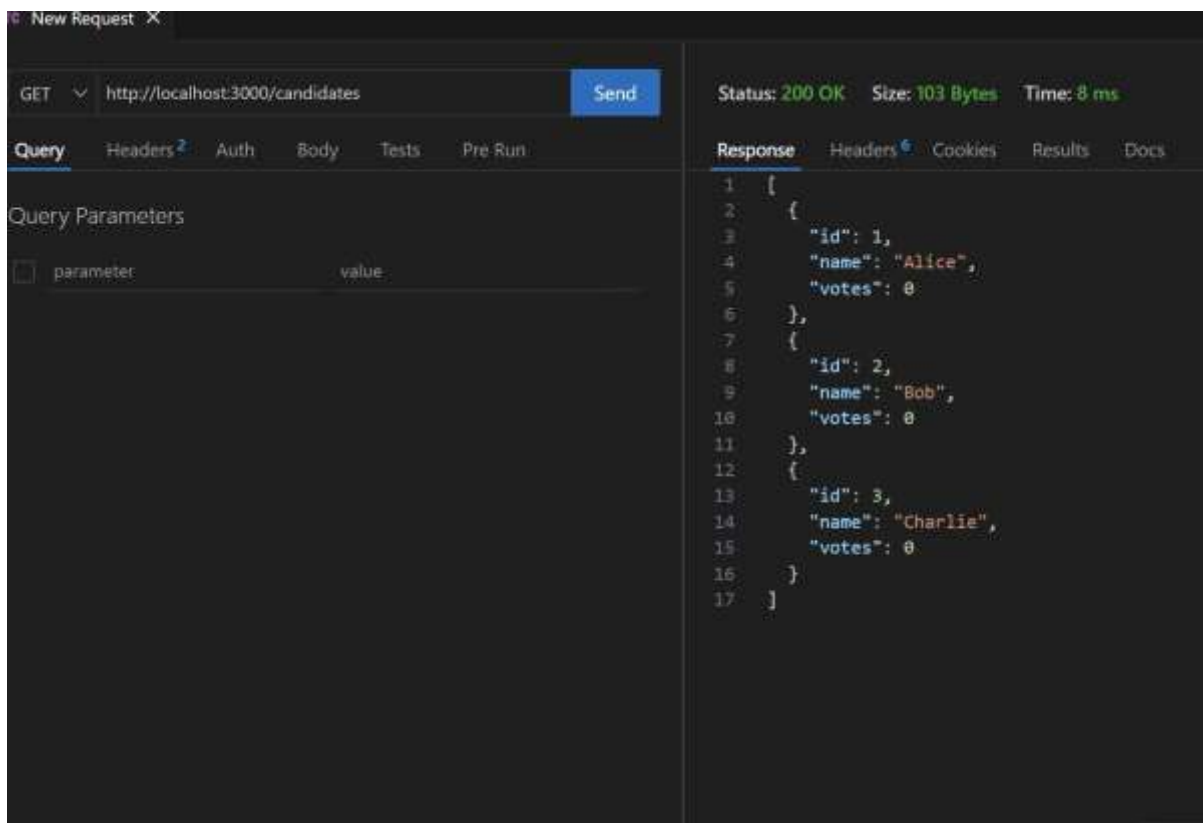


Task 4 – Online Voting API



```
1 // Import express
2 const express = require("express");
3 const app = express();
4
5 app.use(express.json());
6
7 // Candidate data
8 let candidates = [
9   { id: 1, name: "Alice", votes: 0 },
10  { id: 2, name: "Bob", votes: 0 },
11  { id: 3, name: "Charlie", votes: 0 }
12 ];
13
14 // GET all candidates
15 app.get("/candidates", (req, res) => {
16   res.json(candidates);
17 });
18
19 // POST vote for candidate
20 app.post("/vote", (req, res) => {
21
22   const candidate = candidates.find(c => c.id == req.body.id);
23
24   if (!candidate) {
25     return res.json({ message: "Candidate not found" });
26   }
27
28   candidate.votes++;
29
30   res.json({
31     message: "Vote recorded"
32   });
33 });
```

o **GET /candidates** → **List candidates**



New Request X

GET Send

Status: 200 OK Size: 103 Bytes Time: 8 ms

Query Headers Auth Body Tests Pre Run

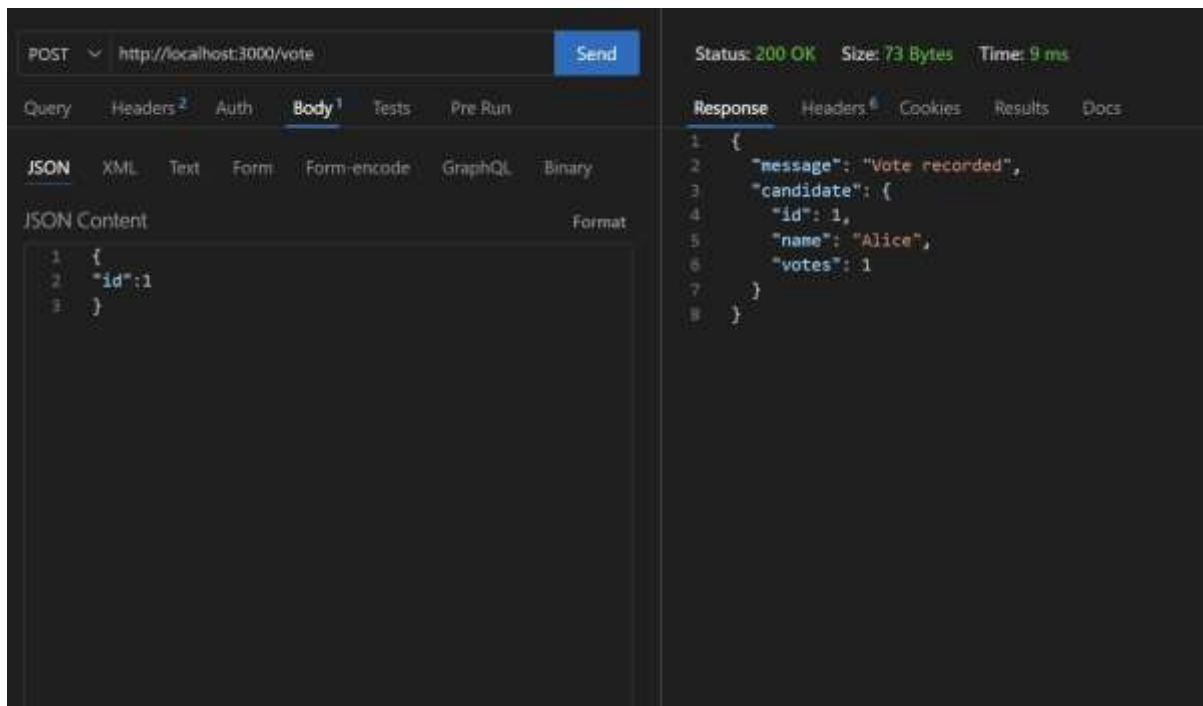
Response Headers Cookies Results Docs

Query Parameters

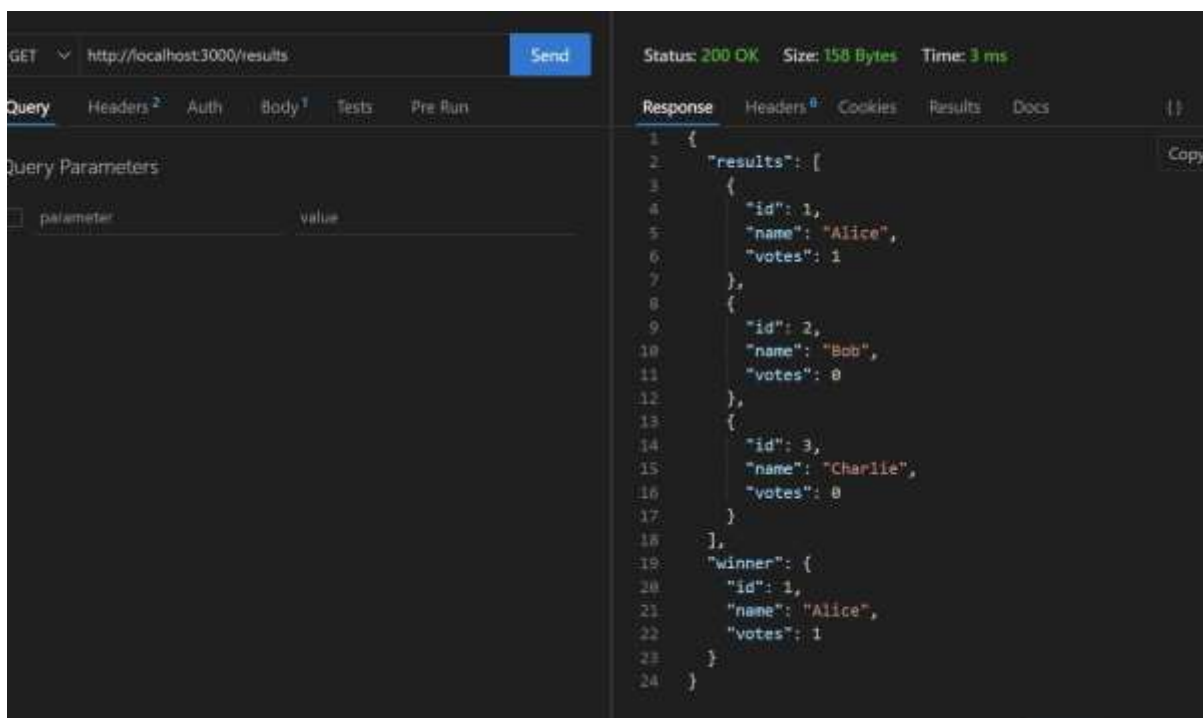
parameter	value
-----------	-------

```
1 {
2   {
3     "id": 1,
4     "name": "Alice",
5     "votes": 0
6   },
7   {
8     "id": 2,
9     "name": "Bob",
10    "votes": 0
11  },
12  {
13    "id": 3,
14    "name": "Charlie",
15    "votes": 0
16  }
17 }
```

o **POST /vote** → **Cast a vote**



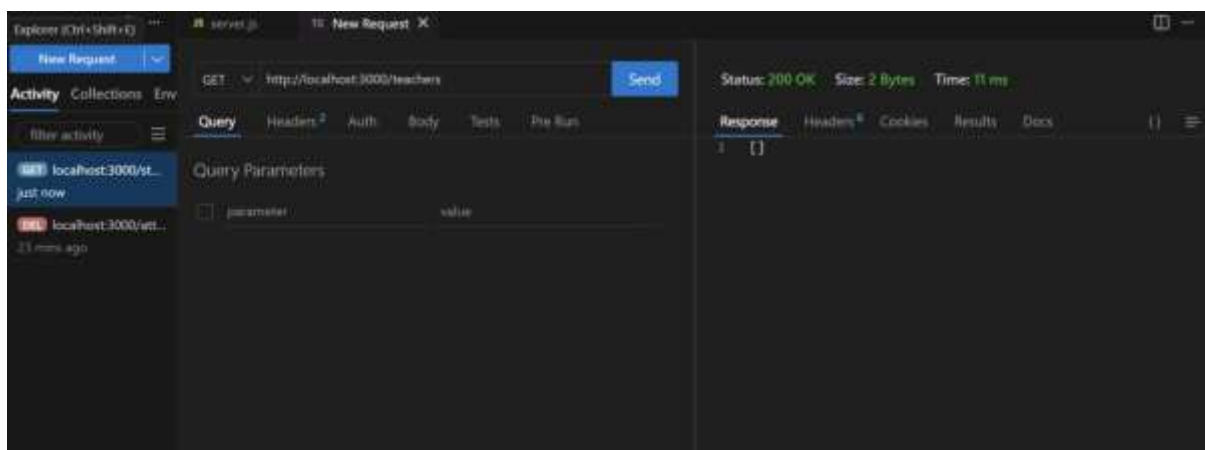
o GET /results → View voting results



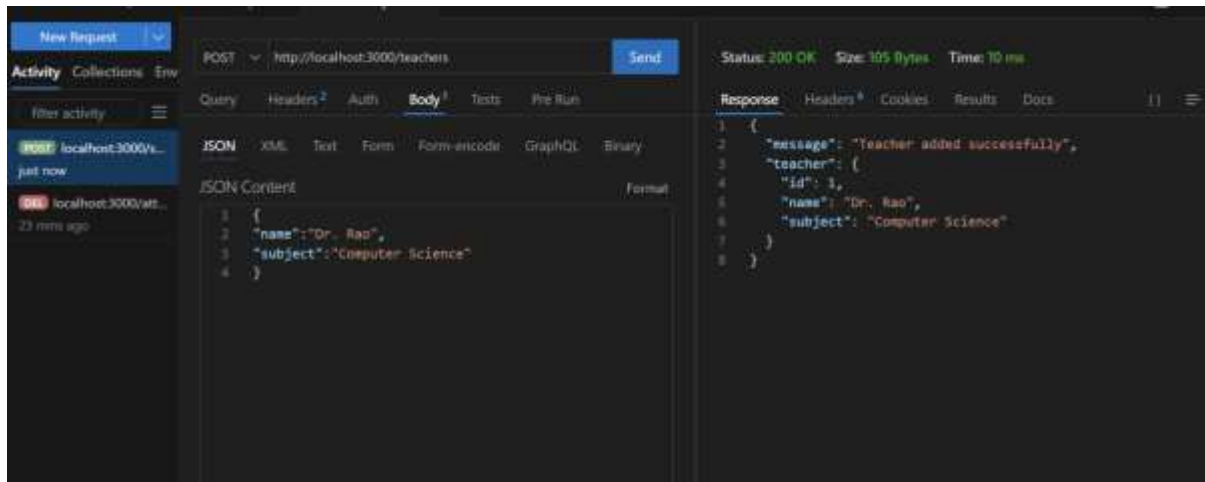
Task 5 – Teacher Management API

```
JS server.js x TC New Request
JS server.js > @app.put("/teachers/:id") callback > message
37 app.put("/teachers/:id", (req, res) => {
45   teacher.name = req.body.name || teacher.name;
46   teacher.subject = req.body.subject || teacher.subject;
47
48   res.json({
49     message: "Teacher updated",
50     teacher: teacher
51   });
52 });
53
54 // DELETE /teachers/:id = Remove teacher
55 app.delete("/teachers/:id", (req, res) => {
56   const teacher = teachers.find(t => t.id == req.params.id);
57
58   if (!teacher) {
59     return res.status(404).json({ message: "Teacher not found" });
60   }
61
62   teachers = teachers.filter(t => t.id != req.params.id);
63
64   res.json({ message: "Teacher removed successfully" });
65 });
66
67 // Start server
68 app.listen(3000, () => {
69   console.log("Server running on port 3000");
70 });
71
```

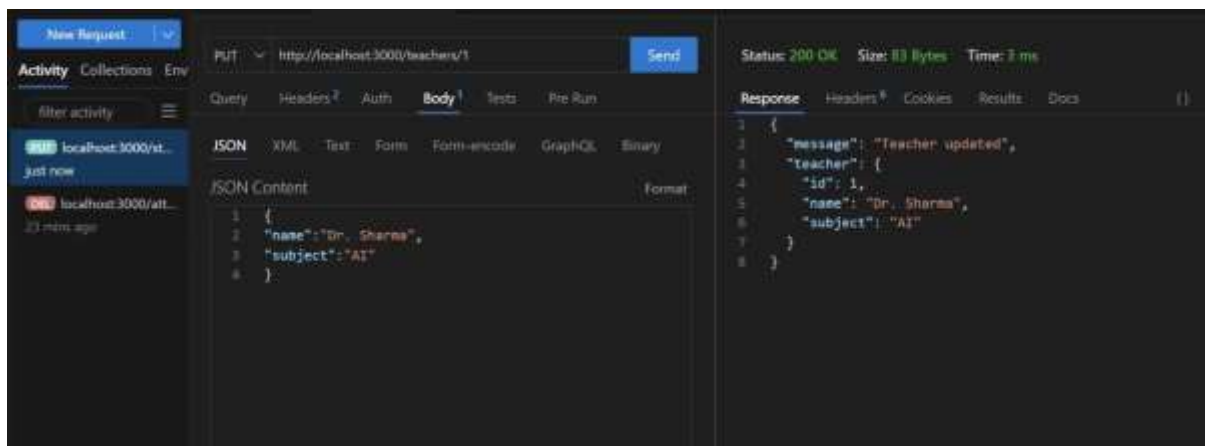
o GET /teachers → List all teachers



o POST /teachers → Add a new teacher



o PUT /teachers/{id} → Update teacher details



o DELETE /teachers/{id} → Remove a teacher

